

Analyze the impact of recursion overhead in divide and conquer algorithms using Binary Search and Merge Sort. Record execution time and stack usage. Compare recursive overhead with iterative approaches. Interpret results and justify trade-offs between readability and efficiency.

Input code:

```
import time
import sys
def binary_search_recursive(arr, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if arr[mid] == key:
        return mid
    elif key < arr[mid]:
        return binary_search_recursive(arr, low, mid - 1, key)
    else:
        return binary_search_recursive(arr, mid + 1, high, key)
def binary_search_iterative(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]
        merge_sort(L)
        merge_sort(R)
        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
```

```

        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1

            k += 1
        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1
        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1
arr = [5,8,12,15,19,25,31,42,56,60]
key = 31
start = time.perf_counter()
index = binary_search_recursive(arr,0,len(arr)-1,key)
end = time.perf_counter()
print("Recursive Binary Search")
print("Position =", index)
print("Execution Time =", end-start)
start = time.perf_counter()
index = binary_search_iterative(arr,key)
end = time.perf_counter()
print("\nIterative Binary Search")
print("Position =", index)
print("Execution Time =", end-start)
data = [38,27,43,3,9,82,10]
start = time.perf_counter()
merge_sort(data)
end = time.perf_counter()
print("\nMerge Sort")
print("Sorted Array =", data)
print("Execution Time =", end-start)
print("\nApproximate Stack Limit =", sys.getrecursionlimit())

```

Output:

```

Recursive Binary Search
Position = 6
Execution Time = 6.986099998584905e-05

Iterative Binary Search
Position = 6
Execution Time = 0.00042576399999991625

Merge Sort
Sorted Array = [3, 9, 10, 27, 38, 43, 82]
Execution Time = 0.000109754999998266969

Approximate Stack Limit = 1000

```
